

SIF Precursor Detection — SIH 2026 Master Plan (Final)

Pitch: Safety programmes measure how many incidents happen. They don't measure how many could have killed someone. We built the second thing.

1. The Problem

Most safety systems get better at preventing the wrong things. Non-fatal incidents fell 51% over fifteen years while fatalities fell only 25.5%, because low-severity incidents and fatalities don't share the same causes. Roughly 20–25% of reports carry genuine fatal (SIF — Serious Injury or Fatality) potential, buried in the pile. We rank and surface those, without ever removing a human from the loop.

2. Naming – Locked, One Set, Everywhere

lsr_rule, hazard_assessment, control_status, severity, is_sif_precursor, flagged_phrases, confidence, reasoning.

Put this exact list in a top-level NAMES.md .
Members 2 and 4 both read it before writing a line of code, in SQL, Pydantic, or the UI. Field names drifted across three separate drafts of this plan before this version – this is the cheapest bug anyone will ever prevent.

3. Team & Ownership

#	Role	Owns	Critical deadline
1	Lead / Product	Rubric (day 1, from PS's own cited sources + IOGP), demo script, pitch, hostile-Q&A defense, second annotator	Rubric day 1

#	Role	Owns	Critical deadline
2	ML	Both classifiers, prompt, Pydantic schema. Baseline before LLM.	—
3	Data	Report generation with metadata, OSHA pull, labelling tool, eval script, first annotator	All 180 reports labelled by day 4–5
4	Backend	FastAPI, Supabase schema, cache, aggregation SQL	Stub endpoint (fake JSON) day 3
5	Frontend	Three screens, live-analyse box on landing page	Builds against stub from day 3
6	DevOps / QA / Research	Deployment, secrets, tests, literature,	Real pytest suite

#	Role	Owns	Critical deadline
		tiebreak annotator	

Two real dependencies, both solved with stubs:

Member 3 gates Members 2 and 4 via labels (fake labels file by day 2). Member 4 gates Member 5 via the API (fake endpoint by day 3). Every other pairing works independently.

4. The Rubric — Three Gates

Written by Member 1, sourced strictly from the PS's own cited literature (DEKRA Martin & Black 2015, the EEI SIF Precursor model, VelocityEHS 2024) plus IOGP Life-Saving Rules for tagging. Member 3 generates reports without seeing the rubric. Neither sees the other's work until both finish.

- **Gate 1 — Hazard.** yes / no / insufficient_information . Is a high-energy hazard present, from the eight IOGP Life-Saving Rules? No or insufficient → stop, not a precursor.

- **Gate 2 – Control.** absent / failed / present / unclear . Present (barrier held) → stop, not a precursor.
- **Gate 3 – Plausible variation.** Would a small, realistic change — different timing, different position, no last-second intervention — have produced death or life-altering injury? This is what separates a missing hard hat at ground level from a missing harness at height. Skip this gate and you over-flag, and your precision suffers for reasons that have nothing to do with the model.

All three yes → SIF precursor.

Quality checkpoint: get a 15-minute external review of the rubric from any EHS/industrial-engineering contact before finalizing it.

Agreement checkpoint: if annotator agreement is below 70% on day 4, the rubric is ambiguous — not the annotators. Revise and re-label. Losing a day here is cheaper than building the rest of the project on labels nobody trusts.

5. Schema (Locked)

```
from pydantic import BaseModel, conint,
confloat
from enum import Enum

class HazardAssessment(str, Enum):
    YES = "yes"
    NO = "no"
    INSUFFICIENT_INFORMATION =
"insufficient_information"

class ControlStatus(str, Enum):
    ABSENT = "absent"
    FAILED = "failed"
    PRESENT = "present"
    UNCLEAR = "unclear"

class LSRRule(str, Enum):
    ENERGY_ISOLATION = "energy_isolation"
    HOT_WORK = "hot_work"
    CONFINED_SPACE = "confined_space"
    LINE_OF_FIRE = "line_of_fire"
    WORK_AT_HEIGHT = "work_at_height"
    LIFTING = "lifting"
    DRIVING = "driving"
    PERMIT_TO_WORK = "permit_to_work"
    NONE = "none"

class ClassificationResult(BaseModel):
    hazard_assessment: HazardAssessment
    lsr_rule: LSRRule
    control_status: ControlStatus | None
```

```
= None
    severity: conint(ge=1, le=5)
    is_sif_precursor: bool
    confidence: confloat(ge=0, le=1)
    flagged_phrases: list[str]
    reasoning: str
```

Why enums, not `bool` | `Literal[...]`: Pydantic's lax bool coercion turns that union into a 2am debugging session.

Why `confidence` is a float, not a sentence: it's what makes PR-AUC computable for the LLM at all.

Why `severity` is back in: it's named directly in the PS and drives the sorted queue in the dashboard.

On validation failure: retry once, then route to human review and **log the rate** — this turns hallucination into a measured number instead of an unbounded risk.

6. Data & Report Metadata

150 synthetic reports, not 90 or 60. At ~22% positive class, a 90-report set leaves roughly six positives in a held-out split — one flipped prediction swings F1 by ~8 points, and your headline number becomes noise.

Every report carries metadata: site, activity, shift, date, contractor flag. Without this, `aggregate.py` — one of the three artifacts that matter most — returns nothing to group by, which is the most likely silent failure in the whole project. Define 8–10 fixed sites up front and distribute reports **unevenly on purpose**: if every site has three precursors, the "Rig 4 had eleven this quarter" demo moment doesn't exist.

Plus 30 real OSHA Severe Injury Reports. Be precise in the pitch about what this proves: that the classifier handles real-world writing nobody on the team produced — not that it detects precursors in near-miss reports. OSHA records describe injuries that already happened, a different population from near-misses, and someone in the room may know that.

Note: OSHA narratives won't carry your site taxonomy, so the aggregation/dashboard view runs on synthetic data only. Say this plainly if asked — it's a fine limitation, not a gap to hide.

Positive class stays 20–25% throughout. Don't rebalance to 50/50 — it flatters every model and convinces nobody experienced.

7. Evaluation – Two Honest Numbers, Not One Misleading One

The baseline is trained on synthetic reports. Testing it against OSHA narratives would measure domain transfer, not model quality – an unfair fight the baseline loses for the wrong reason. Report two separate tables:

- **Synthetic held-out split:** baseline F1 vs LLM F1. This is your fair, apples-to-apples comparison.
- **OSHA set:** LLM F1 only, explicitly framed as a generalization check on unseen real writing.

Your line becomes: *"On matched data, the LLM beats keywords by X. On unseen real writing, it holds most of that."*

Process:

1. Both Member 1 and Member 3 label all 180 reports (150 synthetic + 30 OSHA) independently, days 1–5, no discussion until both finish.
2. Member 6 tiebreaks disagreements.
3. Report plain agreement % and Cohen's kappa (`cohen_kappa_score` – one line, free once labels exist).

4. Split synthetic set: ~30 for dev/prompt-tuning, remainder held out. **Open the held-out set exactly once, at the end.**
5. Baseline gets its own train/test split — say so explicitly.
6. Report **F1 and PR-AUC, never accuracy** (at 25% positives, "always say no" scores 75%).
7. State that you tuned for **recall over precision** as a deliberate design choice: missing a fatal precursor costs a life, a false alarm costs 20 minutes.

Your strongest sentence: *"Two of us labelled independently and agreed 88% of the time. That's the ceiling. Our system hits 89% of it."*

8. Tech Stack (Final)

- **Backend:** Python 3.11, FastAPI + Uvicorn, Pydantic v2 (free schema-failure-rate metric as a byproduct)
- **Database:** Supabase (Postgres, free tier). No pgvector, no embeddings, no clustering. Tables: `reports`, `predictions` (carries `model_version`), `gold_labels` (carries `annotator`), `sites`.

- **Classifiers:** TF-IDF + logistic regression baseline (scikit-learn, ~30 lines); Claude structured-output call as the real classifier. The gap between them is your strongest differentiator — build the baseline first.
- **Explainability:** `flagged_phrases` from the LLM; baseline's top-weighted words via `sorted(zip(vectorizer.get_feature_names_out(), model.coef_[0]))`. No SHAP.
- **Multilingual:** handled natively inside the Claude classification prompt (Hindi/Hinglish/code-mixed) — no separate embedding pipeline.
- **Aggregation:** plain SQL — `GROUP BY site, lsr_rule, control_status HAVING count(*) > n ORDER BY count DESC`. Same dashboard value as clustering, fully explainable by every member.
- **Caching:** SQLite, keyed on `sha256(report_text + prompt_version)` — reproducible eval runs, protects the demo from bad wifi.
- **Offline fallback (new — don't skip this):** the cache only covers reports you've already seen; the live demo box takes new text by definition. On API failure, fall back to the local TF-IDF baseline and say so plainly in the UI. One line of

code, and it's also a strong answer to a resilience question.

- **Frontend:** React + Vite + TypeScript + Tailwind + Recharts.
- **Deployment:** Vercel (frontend), Render (API), both free tier. Secrets via `python-dotenv` locally, Supabase secrets in production. Never commit keys.
- **Testing:** pytest locally, ~10 real tests over the classifier, schema validator, and aggregation query — written *before* claiming they exist in the pitch. No CI — invisible on stage relative to setup cost.
- **Cost:** budget ₹1,000 (₹500 is optimistic once prompt-tuning re-runs are counted). Still effectively free.

9. Repo Layout

```
api/    main.py, schemas.py,
        classifier_llm.py, classifier_base.py,
        aggregate.py, db.py
eval/   label_reports.py, kappa.py,
        run_eval.py
data/   synthetic/, osha/,
        gold_labels.csv, reports_metadata.csv
```

```
web/  
tests/  
NAMES.md
```

The three artifacts that matter most:

1. `eval/run_eval.py` → one table: baseline F1, LLM F1, human agreement ceiling, plus OSHA-set LLM F1 as the generalization check.
2. `api/aggregate.py` → SQL-based precursor density ranking by site/rule/control status.
3. Live text-input demo box on the first screen, never buried in a tab.

10. What All Six Must Be Able to Explain

- **Why not accuracy:** ~1 in 4 reports is serious; "always say no" scores 75% and is useless.
- **Precision vs recall in one breath:** precision = of what we flagged, how much was real; recall = of what was real, how much we caught. We chose recall, deliberately.
- **Why two humans were measured against each other, and what a ceiling means:** if annotators agree 85% of the time, no software can honestly

claim 95% — quoting your own ceiling is the most sophisticated thing a student team can say.

- **Kappa vs raw agreement:** kappa subtracts the agreement you'd expect from chance alone.
- **What happens when the system is wrong:** it never closes a report, only reorders the queue. Every judgement is logged and reviewable.

Never say: "it's a black box," "our accuracy is 89%," "it's 100% accurate on our test set."

Drills, done this week, before any code: the stranger test (explain to someone outside CS), the random-person drill (lead picks who answers, not the owner), the three-why chain (memorized answers collapse by the second why).

11. Role-Specific Hostile Questions

Member 2 (ML): understanding vs pattern-matching (it's sophisticated pattern-matching, and that's enough for a reading-comprehension task) · why not fine-tune (180 examples would overfit; prompting wins at this scale) · what TF-IDF actually does (counts weighted words, no meaning — that's why the gap to the LLM is meaningful) · what

happens on hallucination (schema-constrained, retry once, then human review, rate logged).

Member 3 (Data): "you wrote the reports and graded yourself" (intent labels discarded; two independent annotators against a written rubric; a sixth of the test set is real OSHA text nobody on the team wrote) · why hold out a test set (tuning against your test score fits the model to the answers, not the problem).

Member 4 (Backend): what the SQL aggregation actually computes and why it replaces embeddings for this use case.

Member 1 (Lead): "why can't ChatGPT already do this" (it can classify one report; the product is a consistent schema across thousands of reports, ranked aggregation, and measured human agreement) · why 20–25% carries fatal potential (non-fatal and fatal trends diverged over 15 years — shared causes would mean both fell together).

12. Demo Structure

1. Live analyse box, first screen, not behind a tab.
2. One ambiguous case ready: hazard real, control held → system returns *hazard present, barrier*

present, not a precursor. Proves reasoning over keyword-hunting.

3. Show a misclassification, explain why, explain the fix — hand judges the crack before they find it.
4. Aggregation/dashboard view is the actual product — protect it from being cut for time. *"Rig 4 produced eleven Energy Isolation precursors this quarter, nine during night-shift maintenance"* is a decision, not a demo trick.
5. Close on deployment framing: reorders the queue, never closes a report, every judgement logged and reviewable.

13. Order of Operations

Rubric → reports with metadata (150 synthetic + 30 OSHA) → both annotators label all 180 (days 1–5)
→ agreement/kappa check (revise rubric if <70%)
→ baseline → LLM classifier → two-table
evaluation → aggregation SQL → dashboard → live
demo box → offline fallback → pitch rehearsal
(stranger test + hostile Q&A).

Labels first, always. Everything downstream is meaningless without them.

14. Open Item — Needs an Answer Before Scheduling

The calendar is still unresolved: is 20 September your actual build deadline, or just the submission-form date? Everything in Section 13 assumes ~22 days total, with days 1–5 spent on labelling alone (nearly a quarter of the timeline). If the real deadline is sooner, the labelling window needs to compress and the 70% agreement checkpoint in Section 4 becomes a much tighter risk — worth locking this down before anyone commits to the day-by-day plan.